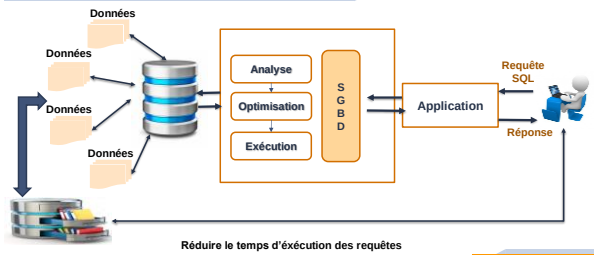
Administration & Tuning des Bases de données

5

Optimisation des requêtes

Problématique



Typologie des requêtes

- **Requête singulière (Point query).**
  - Requête basée sur une seule relation et dont le prédicat de sélection est composé avec une ou plusieurs égalités de manière à ce que **la réponse comprenne au plus un tuple.**

```
SELECT *FROM Customer WHERE ID_Cust= '11111' ;
```

  - **Requête singulière multiple (multipoint query).** Requête basée sur une seule relation dont le prédicat de sélection est composé d'une ou plusieurs égalités, et dont **la réponse comprend plusieurs tuples**

```
SELECT * FROM Customer,
WHERE Ville_Cust = 'Algiers'
AND Gender_Cust ='Female'
```


Typologie des requêtes

- **Requête d'intervalle (range query).** Requête basée sur une seule relation et dont le prédicat comprend un test d'intervalle.

```
SELECT * FROM Customer WHERE age_Cust > 18 AND
age_Cust <30' ;
```

- **Requête min-max (Extermal query).** Requête formulée sur une seule relation et dont le prédicat est composé d'égalités faisant référence à une valeur minimale ou maximale.

```
SELECT Namen, age FROM Customer,
WHERE id_Cust='1111' AND Name="Soumia ; Z = {NSS, Nom}
```

- **Requête d'appartenance à un ensemble ordonné d'attributs Z (prefix match query).** Requête dont les conditions d'égalités doivent inclure obligatoirement une sous-chaîne définie selon l'ensemble Z

```
SELECT * FROM Customer,
WHERE age = MAX(SELECT *age FROM Customer);
```


Typologie des requêtes

- **Requête de groupement (Grouping query).** Requête dont la réponse est partitionnée par un ou plusieurs attributs

```
SELECT AVG(age), Ville_Cust
FROM Customer,
GROUP BY Ville_Cust
```

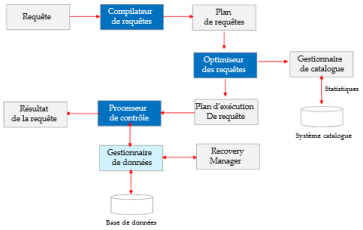
- **Requête de jointure (join query).** Requête basée sur plusieurs relations.

```
SELECT Name, age, FROM Customer C, Sale S
WHERE C. id_Cust = S. id_Cust
```

Traitement d'une requête

- **Réécriture des requêtes :**
  - Transformations à partir d'une requête SQL à une autre en utilisant des propriétés sémantiques.
- **Sélection du plan d'exécution de la requête :**
  - Fait unique sur blocs de requêtes
  - **Étape principale :** l'énumération de jointures
- **Estimation de coût:**
  - Pour comparer entre les plans dont nous avons besoin pour évaluer leur coût en utilisant ***des statistiques sur la base de données***.

Etapes de traitement d'une requête



Etapes de traitement d'une requête

- **Compilateur de requête: Analyse des requêtes et sa traduction**
  - vérifier la syntaxe (par exemple SQL pour SGBD relationnel)
  - vérifier que les relations mentionnées existent et remplacent vues
  - Transformer la requête SQL à un plan de requête représentée par l'expression de l'algèbre relationnelle (pour SGBD relationnel) en utilisant les différentes expressions possibles d'algèbre relationnelle pour une seule requête

Etapes de traitement d'une requête

- **Optimiseur des requêtes: Optimisation des requêtes**
  - transformer le plan de requête initiale dans le meilleur plan de requête possible sur la base de l'ensemble de données
  - Préciser l'exécution des opérations du plan simple de requête (primitives d'évaluation)
    - ***Par exemple*** *qui algorithmes et indexes utilisés*
  - Le plan d'exécution de requête est défini par une séquence de primitives d'évaluation

Etapes de traitement d'une requête

- **Processeur de commandes : Evaluation de la requête**
  - exécuter le plan d'exécution de la requête et de retourner le résultat

Optimisation des requêtes

- L'optimisation des requêtes consiste à choisir une stratégie d'exécution efficace pour le traitement de la requête.
- Il y aura de nombreuses transformations équivalentes de haut niveau pour la même requête. Le but de l'optimisation est de **choisir un plan d'éxecution qui minimise l'utilisation des ressources**.
- Le problème est difficile avec un grand nombre de relations, de sorte que la stratégie adoptée est réduite à trouver une proche solution optimale.

Optimisation des requêtes

- **Plans d'exécutions équivalents**
  - Transformer la requête SQL pour le plan de requête suivante

```
SELECT name, street
FROM Customer, Order
WHERE Order.customerID = Customer.customerID AND
status = 'open';
```

$$\pi_{\text{name, street}}(\sigma_{\text{status="open"}}(\text{order} \bowtie \text{customer}))$$

Idéalement :
trouver le meilleur plan.
Pratiquement :
Éviter le pire plans!

Optimisation des requêtes

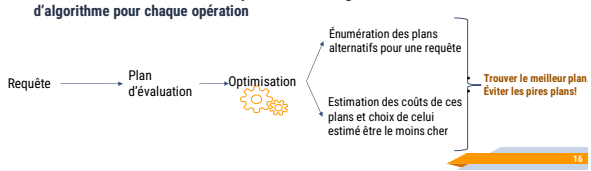
- **Chercher le meilleur plan d 'exécution?**
  - coût excessif
- **Solution approchée à un coût raisonnable**
  - Générer les alternatives heuristiques
- **Choisir la meilleure**
  - estimation approximative du coût

Traitement de la requête

- **Requêtes exprimées en SQL:** langage déclaratif, on indique ce que l'on veut obtenir, on ne dit pas comment l'obtenir
- **Le SGBD doit faire le reste**
  - Déterminer la façon d'exécuter la requête: **plan d'exécution**
  - Plusieurs plans possibles, choisir le meilleur :
    - optimisation
    - Exécuter le plan choisi: évaluation
- **Plan d'exécution**
  - **Exprimé en algèbre relationnelle** (expression algébrique)
  - **Forme exécutable:** on sait précisément comment l'évaluer

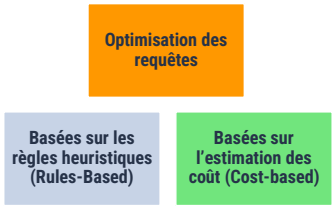
Résumé

- **Les étapes d'évaluation d'une requête SQL**
  - Analyse syntaxique de la requête
  - Transformation de la requête en plan d'évaluation
- **Plan d'évaluation c'est arbre d'opérations de l'algèbre relationnelle avec un choix d'algorithme pour chaque opération**



Optimisation des requêtes

- **Techniques**



Optimisation des requêtes à base de règles

Rule-based Optimization Approach (RA)

Ces méthodes s'appuient sur les propriétés algébriques des opérateurs qui permettent de limiter la taille des relations manipulées

- **Problème**
  - suivant l'ordre des opérateurs algébriques dans un arbre, le coût d'exécution est varie.
- **Pourquoi?**
  - 1. le coût des opérateurs **varie** en fonction du volume des données traitées
    - i.e., plus le nombre de tuple des relations traitées est petit, plus les coûts CPU et d'E/S sont minimisés
  - 2. certains opérateurs diminuent le volume des données comme restriction et projection

19

Rappel: Algèbre relationnelle

Algèbre relationnelle

- Modèle formel permettant d'exprimer et de calculer les requêtes et les relations
- Constituée d'un ensemble d'opérateurs algébriques: la sélection, la jointure la projection et les opérateurs ensemblistes
- Le résultat de l'exécution d'un opérateur est toujours une relation, ce qui permet la composition
- Utilisée également pour l'optimisation de requêtes
- Une requête algébrique peut se présenter sous forme d'un arbre algébrique

20

Rappel: Arbre algébrique

Arbre algébrique

- Permet de représenter une requête sous forme graphique d'arbre plus facile à lire.
- **Les nœuds feuilles** sont les **relations de base**
- Les **nœuds intermédiaires** sont les **opérateurs**
- Le **nœuds racine** est le **résultat**
- **L'arc** est un **flux** de données

21

Arbre algébrique : Exemple

- **Propriétaire** (numPropriétaire, prénom, nom, adresse, numtél).
- **Propriété** (numPropriété, ville, type, location, numPropriétaire*)
- **Client** (numClient, prénom, nom, adresse, typepréf, locMax)
- **Visite** (numPropriété*, numClient*, datevisite)

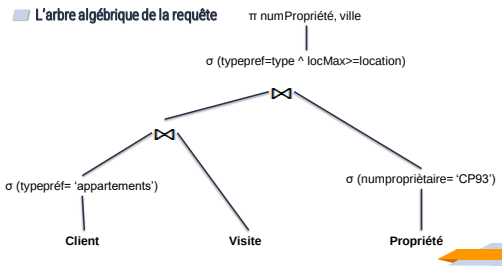
Exprimer en SQL la requête suivante : pour le compte de locataires potentiels qui recherchent des *appartements*, trouver tous les logements qui correspondent à leurs exigences *(locMax, typepréf)* et qui appartiennent au propriétaire de *numéro CP93*.

- **SELECT** P.numPropriété, P.ville
- **FROM** Client C, Visite V, Propriété P
- **WHERE** C.typepréf=' appartements'
- **AND** C.typepré= P.type AND C.locMax>= P.location **AND** P.numPropriétaire=' CP93'
- **AND** C. numClient=V. numClient **AND** V. numPropriété=P. numPropriété

22

Exemple d'un arbre algébrique:

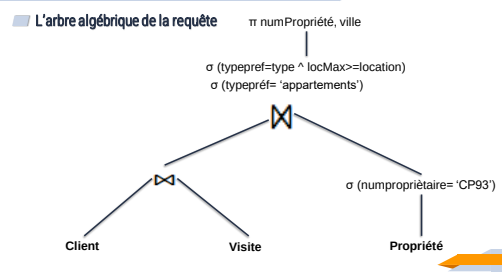
- L'arbre algébrique de la requête



23

Exemple d'un arbre algébrique:

- L'arbre algébrique de la requête



24

Méthodes d'optimisation basées sur les arbres algébriques

Méthodes d'optimisation basées sur les arbres algébriques

Restructuration algébrique

- Pour une requête SQL, il y a plusieurs expressions algébriques équivalentes possibles
- On passe d'une expression à une autre équivalente en utilisant **des règles de réécriture**

Le rôle de l'optimiseur:

- Trouver l'expression algébrique optimale (temps d'exécution minimal) en appliquant les règles de réécriture

Rules Based Optimisation

Requête (SQL)

↓ Traduction

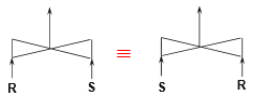
Requête en algèbre

↓ Transformation en fonction des propretés des opérateurs

Plan optimal

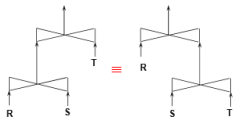
Principe de reconstruction

- 1. Commutativité pour jointure et produit



Principe de reconstruction

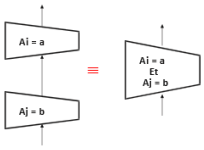
- 2. Associativité pour jointure et produit



- (R join S) join T = R join (S join T)
- Il existe N/2 arbres de jointure de N relations. ·

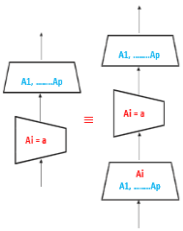
Principe de reconstruction

- 3. Groupage des sélections
  - σF1 (σF2 (E)) = σ F1 ∧F2(E)
  - **Exemple :** σage>20 (σgenre'F' (Étudiant)) = σ(age>20) ∧(genre ='F')(Étudiant)



Principe de reconstruction

- **4. Semi-commutativité des projections et restriction**
  - Il est possible de descendre les projections, mais les attributs utilisés dans la suite doivent être conservés !!!



Principe de reconstruction

- **5. Groupage des projections**
  - $\pi A_1, \ldots, A_n (\pi B_1, \ldots, B_m (E)) = \pi A_1, \ldots, A_n (E)$ --- $\{A_1, \ldots, A_n\}$ inclus dans $\{B_1, \ldots, B_m\}$
- **6. Inversion σ et π**
  - $\pi A_1, \ldots, A_n (\sigma F(E)) = \sigma F(\pi A_1, \ldots, A_n (E))$ si F porte sur A1, …, An
  - $\pi A_1, \ldots, A_n (\sigma F(E)) = \pi A_1, \ldots, A_n (\sigma F(\pi A1, \ldots, An, B1, \ldots, Bn) (E))$ si F porte sur B1, …, Bm qui ne sont pas parmi A1, …, An

Principe de reconstruction

- **7. Inversion Sélection-Produit**
  - $\sigma_F (E1 * E2) = \sigma_F (E1) * E$ si F ne porte que sur les attributs de E1
  - $\sigma_F (E1 * E2) = \sigma_{F1} (E1) * \sigma_{F2} (E2)$ Si F = F1 ∧F2 et Fi ne porte que sur les attributs de Ei
  - $\sigma_F (E1 * E2) = \sigma_{F2} (\sigma F1 (E1) * E2)$ si F1 porte sur les attributs de E1 et F2 sur ceux de E1 et E2
  - **Exemple :** σGenre ='F'(Étudiant * Cours) = σGenre='F'(Étudiant) * Cours
- **8. Inversion Sélection-Union**
  - $\sigma_F (E1 \cup E2) = \sigma_F (E1) \cup \sigma_F (E2)$

Principe de reconstruction

- 9. Inversion Sélection-Diffétrence
  - σF (E1 - E2) = σF(E1) - σF(E2)
- 10.Inversion Projection-Produit
- 11. Inversion Projection-Union

37

Optimisation des requêtes à base de règles

- **Principales règles de Restructuration**
  - Semi-commutativité des restrictions et jointures
  - Semi-distributivité des projections / jointures
  - Distributivité des restrictions / unions ou différences
  - Distributivité des projections / unions

38

Principe d'optimisation des expressions algébriques

1. Exécuter les sélections aussitôt que possible
2. Réduire la taille des relations à manipuler
   - Combiner les sélections avec un produit cartésien pour aboutir à une jointure
3. Combiner des séquences d'opérations unaires (σ, π)
4. Mémoriser les sous-expression commune

39

Optimisation des requêtes à base de règles

- **Heuristique d'Optimisation**
  - Appliquer d'abord les opérations réductrices (restrictions et projections) en les groupant sur chaque relation.
    - 1. Dégrouper les restrictions
    - 2. Descendre les restrictions
    - 3. Grouper les restrictions aux feuilles
    - 4. Descendre les projections
- L'ordre des unions, différences et jointures reste inchangé !!!

40

Algorithme d'optimisation

1. Séparer chaque sélection **σF1 ∧… ∧Fn(E)** en une cascade **σF1 (σF1 … (σFn(E)))**
2. Descendre chaque sélection le plus bas possible dans l'arbre algébrique
3. Descendre chaque projection aussi bas possible dans l'arbre algébrique
4. Combiner des cascades de sélection et de projection dans une sélection seule, une projection seule, ou une sélection suivie par une projection
5. Regrouper les nœuds intérieurs de l'arbre autour des opérateurs binaires (*, -, ∪). Chaque opérateur binaire crée un groupe
6. Produire un programme comportant une étape pour chaque groupe, à évaluer dans n'importe quel ordre mais tant qu'aucun groupe ne soit évalué avant ses groupes descendants.

41

Principe d'optimisation des expressions algébriques

Exécuter les sélections aussitôt que possible: consiste à:

- pousser (tant que possible) les restrictions vers le fond de l'arbre algébrique (c'est-à-dire vers les feuilles de l'arbre).
- Appliquer les règles suivantes (remplacer membre gauche par droit)
  - **1.** σ_F (E1 ∪E2) = σ_F (E1) ∪σ_F (E2)
  - **2.** σ_F (E1 - E2) = σ_F (E1) - σ_F (E2)
  - **3.** σ_F (E1 op E2) = σ_F (E1) op E2 si F ne porte que sur les attributs de E1
  - **4.** σ_F (E1 op E2) = σ_{F1} (E1) op σ_{F2} (E2) si F = F1 ∧F2 et Fi ne porte que sur les attributs de Ei
  - **5.** σ_F (E1 op E2) = σ_{F2} (σ_{F1} (E1) op E2) si F porte sur les attributs de E1 et F2 sur ceux de E1 et E2

  op = * ou ∞

42

Principe d'optimisation des expressions algébriques

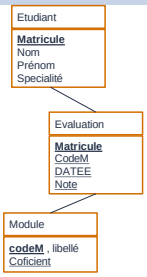
Combiner les sélections avec un produit cartésien pour aboutir à une jointure

- Transformation si possible du produit cartésien en jointure:
  - Si R * S est l'argument d 'une sélection σF alors si F est une comparaison entre attributs de R et de S, transformer R * S en jointure : $\sigma_F (R * S) = (R \bowtie_F S)$
- **Important:** si F est une expression ne portant que sur les attributs de R:
  - $\sigma_F (R * S) = \sigma_F (R) * S$

Sous ensemble communes à une expression

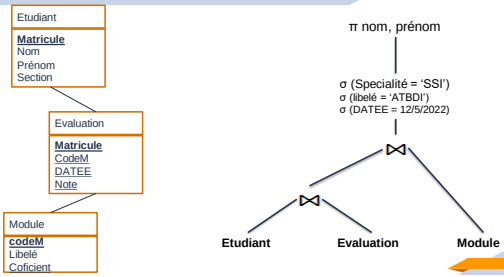
- Si une expression contient plusieurs fois la même sous-expression
- Si cette sous-expression peut être lue en *moins de temps* qu'il faut pour la calculer
- Matérialiser la sous-expression commune
- Principe de vues matérialisées

Exemple



- La liste des étudiants de la spécialité SSI ayant passé l'examen ATBD le 12 Mai 2022

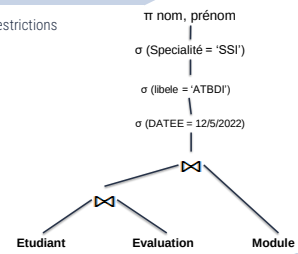
Exemple



46

Exemple

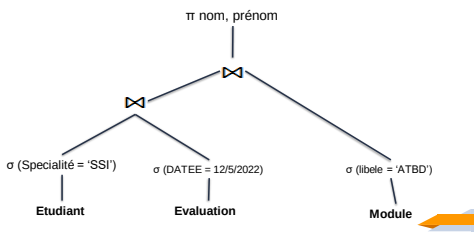
- Phase 1: Décomposer les restrictions



47

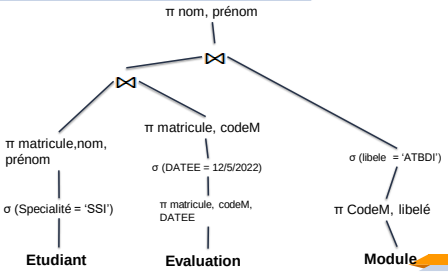
Exemple

- Phase 2: exécuter les restriction le plutôt possible

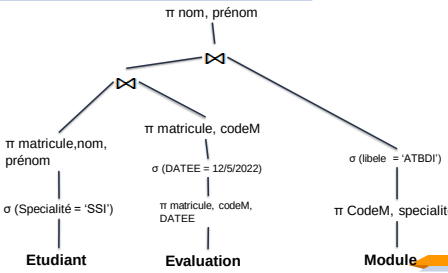


48

Phase 3: exécuter les projections le plutôt possible



Phase 2: exécuter les projections le plutôt possible



Constat

- La réécriture algébrique est nécessaire, mais pas suffisante
- Il faut tenir compte d'autres critères:
  - **Les chemins d'accès aux données**
    - On peut accéder aux données d'une table par accès séquentiel, par index, par hachage, etc.
  - **Les différents algorithmes possibles pour réaliser un opérateur**
    - Il existe par exemple plusieurs algorithmes pour la jointure
    - Souvent ces algorithmes dépendent des chemins d'accès disponibles
  - **Les propriétés statistiques de la base de données**
    - Taille des tables
    - Sélectivité des attributs

Quelques problèmes

- La restructuration d'un arbre sous-entend souvent la permutation des opérateurs binaires
  - Cette optimisation ignore la taille de chaque table, les facteurs de sélectivité, etc.
  - Aucune estimation de résultats intermédiaires
- **Exemple:**

  (Ouvrier ∞ Usine) ∞ Contrat
  - Si cardinalité (Usine) <<<Cardinalité(Contrat) << Cardinalité(Ouvrier) alors l'expression suivante a des chances d'être plus performante:

  (Usine ∞ Contrat) ∞ Ouvrier
- D'où l'orientation vers l'optimisation basée sur les modèles de coût

52

Optimisation des requêtes à base de règles

- **Limites**
  - La restructuration d'un arbre sous-entend souvent la permutation des opérateurs binaires
  - Cette optimisation ignore la taille de chaque table, les facteurs de sélectivité, etc.
  - Aucune estimation de résultats intermédiaires

Optimisation basée sur les modèles de coût (Cost Based COB))

53

Méthodes d'optimisation basées sur les modèles de coûts

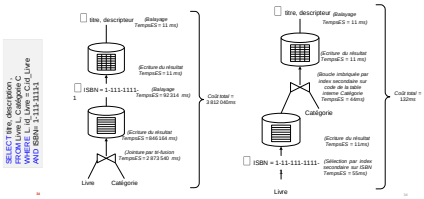
54

Objectif

- Objectif: remplacer les opérateurs algébriques par des algorithmes (appelés également stratégies).
- Le choix de l'algorithme dépend entre autres de la taille des relations (nombre d'accès E/S) et de la prise en compte des chemins d'accès aux relations.

Optimisation des requêtes à base de coût

- **Motivation**



Optimisation des requêtes à base de coût

- **Estimation de l'évaluation du coût d'un Plan**
  - ***Pour chaque plan***, nous devrions être en mesure d'estimer ***le coût global***.
  - ***Pour chaque nœud de l'arbre de requête***,
    - nous estimons le coût d'exécution de l'opération correspondante.
    - nous estimer la taille du résultat, qui est utilisée par l'opération sur le nœud parent.
  - Afin d'estimer correctement le coût de chaque opération et de la taille de son résultat, l'optimiseur utilise certaines informations ***statistiques gérées par le SGBD courant***.

Optimisation des requêtes à base de coût

- **Estimation de l'évaluation du coût d'un Plan**
  - Les coûts d'interrogation sont définis par ***le temps de réponse une requête*** (traiter le plan d'exécution de la requête)
  - Différents facteurs contribuent aux coûts de la requête
    - le temps d'accès au disque, le temps de CPU ou même le temps de communication de réseau
  - Les coûts sont souvent dominés par le temps d'accès au disque
    - temps de recherche (TS) (~ 4 ms)
    - temps de transfert (TT) (par exemple 0,1 ms par bloc disque)
      - Les opérations d'écriture sont normalement plus lent que les opérations de lecture

58

Optimisation des requêtes à base de coût

- **Estimation de l'évaluation du coût d'un Plan**
  - **Simplification**
    - nombre de blocs à transférer
  - **Jointures**
    - taille du résultat
  - **Contexte réparti**
    - seulement coût de transfert entre sites
    - *Hypothèses sur distributions*

59

Notions de base

- Les tables relationnelles sont stockées physiquement dans des fichiers sur disque
  - Le système d'exploitation stocke les enregistrements dans des pages et les pages sont dans des bloc.
  - Le transfère de données entre la MS et la RAM se fait en bloc
- Lorsqu'on veut accéder aux données, on doit transférer le contenu pertinent des fichiers dans la mémoire

60

Notions de base

- CBO (Cost Based Optimisation):
  - Estimation du coût de chaque opérateur
  - prend en charge les caractéristiques des relations et des attributs
- **Fichier =** séquence de tuples
- **Bloc (page) =** unité de transfert de données entre disque et mémoire
- **Facteur de blocage =** nombre de tuples d'une relation qui «tiennent» dans un bloc
- **Coût de lecture (ou écriture) d'une relation=** nombre de blocs à lire du disque vers la mémoire (ou à écrire de la mémoire vers le disque)

61

Estimation du cout des opérations physiques

- La performance d'une requête est évaluée en fonction:
  - **TempsES :** temps d'accès (lecture et écriture) à mémoire secondaire (MS) exprimé en nombre de pages chargés
  - **TempsUCT :** Souvent négligeable (temps de traitement au niveau de la RAM)
  - **TailleMC :** espace mémoire centrale
  - **TailleMS :** espace mémoire secondaire
- Il s'agit d'une estimation du coût des opérations physiques

62

Modèle de coût d'une entrée-sortie en mémoire secondaire

Paramètre	Signification
TempsESDisque(n)	Temps total de transfert (lecture ou écriture) de n octets du disque
TempsTrans(n)	Temps de transfert des n octets sans repositionnement
TempsPosDébut	Temps de positionnement au premier octet à transférer (ex : 10 ms)
TempsRotation	Délai de rotation (ex : 4 ms)
TempsDépBras	Temps de déplacement du bras (ex : 6 ms)
TauxTransVrac	Taux de transfert en vrac (ex : 2MB/sec) Le transfert en vrac (ou en anglais "bulk transfer") est une technique de transfert de données qui consiste à transférer de grandes quantités de données en une seule fois plutôt que de les transférer une par une.
TempsESBloc	Temps de transfert d'un bloc (ex : 11 ms)
TempsTrans	Temps de transfert d'un bloc sans repositionnement (ex : 1 ms)
TailleBloc	Taille d'un bloc (ex : 2K octets)

TempsESBloc = TempsESDisque(TailleBloc) = TempsPosDébut + TempsTrans (TailleBloc)
TempsTrans (TailleBloc) = TailleBloc / TauxTransVrac

63

Statistiques au sujet des tables

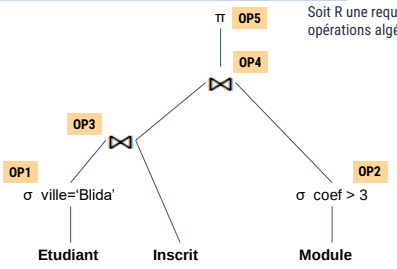
- L'estimation du coût des opérations se base sur des statistiques d'une table T:

Statistique	Signification
‖T‖	Nombre de lignes de la table T
TailleLigne	La taille d'une ligne de la table T
FBT	Facteur de blocage moyen de T (nombre moyen de lignes contenues dans un bloc
NDIST(T,a)	Nombre de valeurs distinctes de la colonne a pour la table T
MinT(colonne)	Valeur minimum de la colonne de T
MaxT(colonne)	Valeur maximum de la colonne de T
| T |	Nombre de pages de la table T
FBM_T	Facteur de blocage maximal de T , **Estimation** : (TailleBloc-TailleDescripteurBloc)/TailleLigneT
B_T	Nombre de blocs de la table T, **Estimation : ‖ T ‖ / FBT**

Les données statistiques

- Pour optimiser la requête, **l'optimiseur utilise les données statistiques de la BD** mémorisées dans les catalogues telles que :
  - la cardinalité des relations,
  - le nombre de pages occupées par une relation,
  - le nombre de niveaux d'index,
  - le nombre de pages de chaque niveau, etc

Coût d'exécution d'une requête

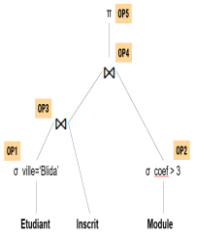


Soit R une requête composée de n opérations algébriques (Opi)

Coût d'exécution d'une requête

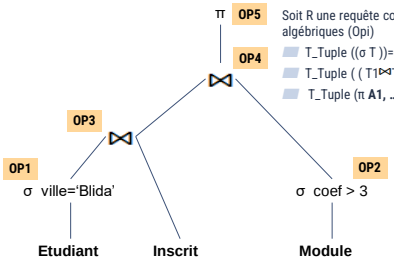
- Soit **R** une requête composée de n opérations algébrique **(Opi)**
- **Coût (R) =∑ coût (Opi)**
- **Coût (Opi) = Nombre de pages transférées de la MS vers la RAM ou de la RAM vers la mémoire swap**
- **Pour calculer le coût (Opi +1)**
  - Il faut calculer le nombre de tuples du résultat intermédiaire de Opi (||Opi||)
    - **En utilisant le facteur de sélectivité de Opi:** || Opi || = FS * || Op_{i-1}||
    - Le transformer en nombre de page en utilisant les tailles du tuples (T_Tuples) et de la page système

      |Opi|=T_tuples * ||Opi|| / PS (PS la taille d'une page système)
    - **Taille tuple=∑** $taille\ attribts\ tuple$
    - **Une autre formule |opi|= ||opi||/ #tuple par page**
- **Si la RAM non suffisante: ajouter les pages transférées en mémoire swap**



67

Coût d'exécution d'une requête



Soit R une requête composée de n opérations algébriques (Opi)

- T_Tuple ((σ T))= T_Tuple (T)
- T_Tuple ((T1⋈T2))= T_Tuple (T1)+ T_Tuple (T2)
- T_Tuple (π **A1, …, An**)= ∑taille (Ai)

68

Facteur de sélectivité

- Le facteur de sélectivité représente le pourcentage de ligne sélectionnées sur le nombre total de tuples
  - Entre 0 et 1
  - **Opération trop sélective:** SF tend vers zéro
  - **Opération non sélective:** SF tend vers 1
- **Exemple:**
  - Si les étudiants habitant **Blida** représentent **20%**
  - Le facteur de sélectivité du prédicat **ville= Blida** est de **0,2 %**
  - Si la table Etudiant contient **100 000 étudiants**, la requête **SELECT * FROM** Etudiant **WHERE** Ville ='Blida' retournera **20 000 tuples**

69

Optimisation des requêtes à base de coût

- **Facteur de sélectivité**
  - Proportion de tuples du produit cartésien des relations touchées qui satisfont une condition.
- Exemple
  - **SELECT** * **FROM** R1, R2 → SF=1
  - **SELECT** * **FROM** R1 **WHERE** A= valeur → SF= 1/NDIST (A) avec un modèle uniforme
- FS d'une projection = 1

70

Optimisation des requêtes à base de coût

- **Facteur de sélectivité**
  - Proportion de tuples du produit cartésien des relations touchées qui satisfont une condition.
- **Exemple**

```
SELECT * FROM Customer WHERE age_Cust > 18;
```

FS = 1/NDIST(A) avec un modèle uniforme

```
SELECT * FROM Customer , Sale ;
```

FS= 1

71

Sélectivité des réstrictions

- **La taille (σ(R) = SF * Taille (R) ie || (σ(R)) || = SF * ||R|| avec**
  - **SF (A = valeur)** = 1 / NDIST(A)
  - **SF(A > valeur)** = (max(A) - valeur) / (max(A) - min(A))
  - **SF(A < valeur)** = (valeur - min(A)) / (max(A) - min(A))
  - **SF(A IN liste valeurs)** = (1/NDIST(A)) * CARD(liste valeurs)
  - **SF(P et Q)** = SF(P) * SF(Q)
  - **SF(P ou Q)** = SF(P) + SF(Q) - SF(P) * SF(Q)
  - **SF(not P)** = 1 - SF(P)
- **Le coût dépend de l'algorithme (index, hachage ou balayage).**

72

Statistiques d'évaluation

Statistiques d'une expression de sélection par valeur (col = val) ou par intervalle (col ∈[val1, val2]) :

- **FacteurSélectivitéT** (col = val): Pourcentage de lignes pour lesquelles la colonne col contient la valeur val,
  - **Estimation : 1/NDIST (col)**
- **FacteurSélectivitéT** (col ∈[val1,val2]): Pourcentage de lignes pour lesquelles la colonne col contient une valeur comprise entre val1 et val2,
  - **Estimation : (val2- val1)/(MaxT (col)- MinT (col))**
- **SelT (col = val):** Nombre de lignes de T sélectionnées par l'expression de sélection,
  - **Estimation : FacteurSélectivitéT (col) * || T ||**

73

Sélectivité d'une jointure

- La taille d'une jointure est estimée par la formule suivante:
  - **|| R1⋈ R2 || = Sel * ||R1|| * ||R2||;**
  - avec **Sel**: la sélectivité de la jointure, il depend de la corrélation des colonnes
    - **Sel = 0** si aucun tuple ne joint
    - **Sel = 1 / MAX(NDIST(A),NDIST(B))** si distribution uniforme équiprobable des attributs A et B sur un même domaine
    - **Sel = 1** si produit cartésien
- Jointure sur clé étrangère sur R1**: | | R1 ⋈ R2 | | = | | R1 | |**
- Une estimation de Sel si jointure quelconque:
  - **SF(R1 $_a$⋈$_b$ R2 | | = 1/ Max (NDIST (a), NDIST (b))**

74

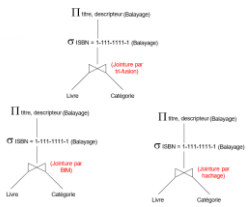
Algorithmes généraux pour les opérations relationnels

- L'algorithme change radicalement les coûts
  - linéaire si index,
  - produit des tailles si boucles imbriquées.
- Opérations à optimiser
  - **1) Sélection de lignes selon une clé**
    - Ex: balayage, index, hashage, etc.
  - **2) Tri des lignes d'une table**
    - Ex: algorithme tri-fusion
  - **3) Jointure de deux tables**
    - Ex: boucles imbriquées, jointure tri-fusion, etc

75

Optimisation des requêtes à base de coût

- **Principaux algorithmes**
  - **Jointure sans index**
    - Le plus simple : jointure par boucles imbriquées
    - Le plus courant: jointure par tri-fusion
    - Par fois le meilleur: jointure par hachage
  - **Jointure avec index**
    - Avec un index: jointure par boucles indexée.
    - Avec deux index: on fait comme si on avait un seul index



Algorithmes généraux pour les opérateurs relationnels

La sélection

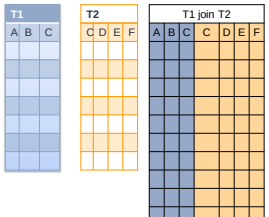
- **Sélection sans index**
  - **SELECT** NSS, Nom **FROM** Employé **WHERE** Salaire > 15000
- **Plan d'exécution**
  - pour chaque tuple t de l' Employé faire -- accès séquentiel
  - si t.salaire > 15000 alors réponse = réponse U πNSS, Nom (Employé)
- Coût = | T | = | Employé | **=T_tuples * ||Opi|| / PS**
- **Sélection avec index**
  - Charger l'index: coût =| index |
  - Parcourir l'index en RAM, chercher les tuples à sélectionner: **coût =0**
  - Charger les tuples sélectionnées: **coût= FS (salaire > 15000) * | Employé |**
  - Total= |index | + FS |T |

- En cas d'absence de statistique, les estimations considèrent une distribution uniforme.

Les jointures

Algorithme de jointure

- **Jointure sans index**
  - Jointure par boucles imbriquées (nested loop)
  - Jointure par tri-fusion (sort-join)
  - Jointure par hachage (hash-join)
- **Jointure avec index**
  - Jointure avec boucles indexées



1. Jointure par boucle imbriquées

- **R ∞F S**
- **Principe:**
  - La première table est lue séquentiellement et de préférence stockée entièrement en mémoire
  - Pour chaque tuple de R, il y a une comparaison avec les tuples de S
- **Algorithme**

Pour chaque ligne IR de R
  Pour chaque ligne de IS de S
    Si θ sur IR et IS est satisfait
      produire la ligne concaténée à partir de IR et IS
    FSI
  Fin Pour
Fin Pour

La boucle externe permet de parcourir les ligne de la 1ère table
Pour chaque ligne de la première table, on charge les tuple de la 2ème table

1. Jointure par boucle imbriquées

θ *la condition de la jointure*

- **Algorithme**

Pour chaque ligne **IR** de **R**
  Pour chaque ligne de **IS** de **S**
    Si θ sur **IR** et **IS** est satisfait
      produire la ligne concaténée à partir de **IR** et **IS**
    FSI
  Fin Pour
Fin Pour

- **Algorithme simple**
  - Cout d'exécution de la jointure: (| **R** | + | **R** | * | **S** |)
  - Si l'une des tables peuvent être chargé entièrement en RAM: | **R** | + | **S** |

1. Jointure par boucle imbriquées

```
Pour chaque bloc B1 de R1 faire
    Lire (R1, B1) ;
      Pour chaque bloc B2 de R2 faire
          Lire (R2, B2) ;
          Pour chaque tuple tuple1 de R1 faire
              Pour chaque tuple tuple2 de R2 faire
                  Si tuple1 et tuple2 sont joignable alors écrire (Resultat, tuple1+tuple2) ;fin
              fin Pour
          fin pour
      fin pour
Fin pour
```

1. Jointure par boucle imbriquées

Opérateur de jointure : boucles imbriquées

- Cout(opération)=B_{R1}+(B_{R1} * B_{R2})

  (on suppose qu'on a en mémoire 2 bloc: 1 bloc pour chaque table)

- Si la mémoire permet de mémoriser (B+1) blocs, il est possible de garder B blocs de R1 et de lire R2 seulement BR1/B fois, la formule devient alors:

  Cout(opération)= B_{R1} +((B_{R1} / B) * B_{R2})

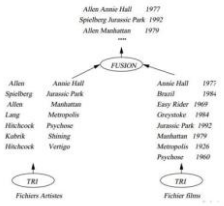
- Si R1 entre en mémoire alors le coût est B_{R1} + B_{R2}

1. Jointure par boucle imbriquées

- La jointure par boucles imbriquées est inefficace sur de grandes tables
- Toutefois, si l'une des relations entre en mémoire , elle est très efficace

2. Jointure par trie de fusion

- **Plus efficace que les boucles imbriquées pour les tables volumineuses**
  - **Plus efficace que les boucles imbriquées pour les grosses tables**
- **Principe:**
  - Trier les deux tables sur les colonnes de jointure
  - ensuite la fusion des tuples ayant même valeur est effectuée (parcours synchronisé des deux relations) (cas de l'equijointure).
- **Complexité:** Le tri qui coûte cher
- **Important**: on ne peut rien obtenir tant que le tri n'est pas fini



- Jointures par tri-fusion

```
Trier R et S par tri externe et réécrire dans des fichiers temporaires
Lire groupe de lignes GR(cR) de R pour la première valeur cR de clé de jointure
Lire groupe de lignes GS(cS) de S pour la première valeur cS de clé de jointure
TANT QUE il reste des lignes de R et S à traiter
    SI cR = cS
      1.  Produire les lignes concaténées pour chacune des combinaisons de  lignes de GR(cR) et GS(cS);
      2.    Lire les groupes suivants GR(cR) de R et GS(cS) de S;
    SINON
        SI cR < cS
            Lire le groupe suivant GR(cR) de R
        SINON
            SI cR > cS
                Lire le groupe GS(cS) suivant dans S
            FINSI
        FINSI
    FINSI
FIN TANT QUE
```


2. Jointure par trie de fusion

- Cout =
  - **|T1| +|T2|** si T1 et T2 déjà triés
  - 2* |T1| $\log_b$ |T1| **+ |T2| + | T1 |** si T2 triée **(coût de tri de la table T1)**
  - 2* |T2| $\log_b$ |T2| + **|T2| + | T1 |** si T1 triée **(coût de tri de la table T2)**
  - 2* |T1| $\log_b$ |T1| + 2* |T2| $\log_b$ |T2| + **|T2| + | T1 |** si aucune table ni triès
  - b nombre de pages du tempon en mémoire centrale disponible pour le tri

2. Jointure par trie de fusion

- Trier R et S par tri externe et réécrire dans des fichiers temporaires
- Lire groupe de lignes GR(cR) de R pour la première valeur cR de clé de jointure
- Lire groupe de lignes GS(cS) de S pour la première valeur cS de clé de jointure

TANT QUE il reste des lignes de **R** et **S** à traiter
SI **cR** = **cS**
Produire les lignes concaténées pour chacune des combinaisons de lignes de **GR(cR)** et **GS(cS);**
Lire les groupes suivants GR(cR) de R et GS(cS) de S;
SINON SI **cR < cS**
Lire le groupe suivant GR(cR) de R
SINON SI **cR > cS**
Lire le groupe GS(cS) suivant dans S
FINSI
FINSI
FINSI FIN TANTQUE

1--5	1-3
6--10	4-8
10_15	9-10

- **Performances**
  - Utilisé pour des grandes relations non indexées sur l'attribut de jointure
  - Coût dominé par la phase de tri: Coût = 2 Tris + 1 fusion
    - Coût du tri: $O(B_R \log B_R + B_S \log B_S)$
    - Coût fusion: $O(B_R + B_S)$
  - Intéressant quand l'une des tables ou les deux sont déjà triées
  - Facilite l'élimination des dupliqués ou l'affichage ordonné

92

3. Jointure par hachage

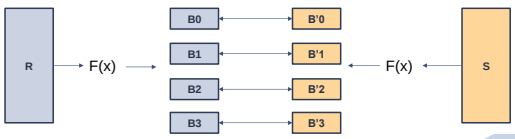
- **Principe:**
  - On hache la plus petite des deux tables en n fragments.
  - On hache la seconde table, avec la même fonction, en n autres fragments.
  - On réunit les fragments par paire, et on fait la jointure.
- L'idée est de partitionner R et S par la même fonction de hachage h, puis de calculer les jointures entre fragments équivalents
- **Important** :pour chaque paire, au moins un fragment doit tenir en mémoire
- La table de hachage d'un fragment doit résider en MC

93

3. Jointure par hachage

Principe

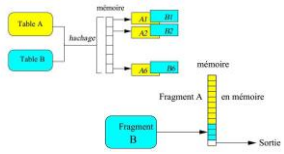
1. Hacher la plus petite des deux tables en n fragments
2. Hacher la seconde table, avec la même fonction, en n autres fragments
3. Réunir fragments par paire, et faire la jointure entre les fragments



94

Optimisation des requêtes à base de coût

Jointure par hachage



3. Jointure par hachage

Les étapes:

1. Charger la relation R en mémoire **| R |**
2. Construire la table de hachage de R (création des groupes en RAM): **0**
3. Déchargement des groupes en MS: **| R |** (chargement des groupes en Mémoire SWAP)
4. Charger la relation S en mémoire **| S |**
5. Construire la table de hachage de S (création des groupes en RAM) : **0**
6. Déchargement des groupes en MS: **| S |** (chargement des groupes en Mémoire SWAP)
7. Chargement du block Bi de R avec B'i de S, faire la jointure entre les deux blocs: **| Bi |+|B'i |**
8. Au total: ∑ | Bi |+|B'i| = | R |+| S |

Cout total = 3 (| R |+| S |)

Contrôle du processus d'optimisation- Cas Oracle

- **Cas Oracle**
  - **Outils**
    - EXPLAIN PLAN
    - SQL Trace
    - Oracle EXPERT
  - HINTS pour forcer l 'utilisation d 'un chemin

```
SELECT /*+ INDEX(EMPLOYÉ INDEX_SEXE)*/
FROM Employé,
WHERE Sexe_Emp= 'F'
```

Contrôle du processus d'optimisation- Cas Oracle

- **Hints**
  - **COST (statistique):** minimise le coût estimé
  - Les directives "hints" sont des moyens de forcer l'optimiseur à choisir un plan d'exécution.
  - Les directives peuvent influencer les plans d'exécution et par conséquent la performance des requêtes

```
SELECT /*+ some hints here)*/
```

- **Exemple**

```
SELECT /*+ INDEX(EMPLOYÉ INDEX_SEXE)*/
FROM Employé,
WHERE Sexe_Emp= 'F'
```


Hints sous Oracle

- **ALTER SESSION SET OPTIMIZER_GOAL =**
  - **RULE**
    - Utilise **l'optimiseur basé sur des règles** (Rule-Based Optimizer).
    - Même si des statistiques existent, elles ne sont pas utilisées.
    - Ce mode est **ancien** et **obsolète**, remplacé par l'optimiseur basé sur les coûts.

Hint	Explication
`CHOOSE`	Utilise le CBO si les statistiques sont disponibles, sinon RBO
`RULE`	Toujours RBO, même si des statistiques existent
`ALL_ROWS`	Optimise pour un **temps total minimal**
`FIRST_ROWS`	Optimise pour un **temps de réponse rapide** (pour les premières lignes)

Hints sous Oracle

- **ALTER SESSION SET OPTIMIZER_GOAL =**
  - **RULE**
    - Utilise **l'optimiseur basé sur des règles** (Rule-Based Optimizer).
    - Même si des statistiques existent, elles ne sont pas utilisées.
    - Ce mode est **ancien** et **obsolète**, remplacé par l'optimiseur basé sur les coûts.
- **Exemple**

```
ALTER SESSION SET OPTIMIZER_GOAL = RULE;

SELECT e.name, d.name
FROM employees e, departments d
WHERE e.dept_id = d.id;
```

  - Oracle suit des règles prédéfinies (ordre de jointure, présence d'index, etc.) sans tenir compte des statistiques

Hints sous Oracle

ALTER SESSION SET OPTIMIZER_GOAL =

- **ALL_ROWS** minimise le temps total (plutôt MERGE JOIN)
  - Utilise l'**optimiseur basé sur le coût** (Cost-Based Optimizer).
  - Objectif est d' **minimiser le temps total d'exécution.**
  - TypConvient aux traitements par lots (batch), où on veut traiter toutes les lignes rapidement.
  - Typiquement, Oracle choisira des stratégies comme **MERGE JOIN (Tri fusion)** .

Exemple

```
ALTER SESSION SET OPTIMIZER_GOAL = ALL_ROWS;

SELECT *
FROM sales
WHERE sale_date BETWEEN TO_DATE('01-01-2024', 'DD-MM-YYYY')
AND TO DATE('31-12-2024', 'DD-MM-YYYY');
```

Oracle peut choisir un **MERGE JOIN** ou un **FULL SCAN** si cela permet d'exécuter toute la requête rapidement.

101

Hints sous Oracle

ALTER SESSION SET OPTIMIZER_GOAL

- **FIRST_ROWS**
  - Utilise également l'optimiseur basé sur le coût.
  - Objectif est de **minimiser le temps de réponse pour les premières lignes**.
  - Utile pour les interfaces interactives ou les applications web.
  - Favorise souvent l'utilisation de **NESTED LOOPS** pour accéder rapidement aux premières lignes.

Exemple

```
ALTER SESSION SET OPTIMIZER_GOAL = FIRST_ROWS;

SELECT *
FROM employees
WHERE department_id = 10
FETCH FIRST 10 ROWS ONLY;
```

Oracle utilisera souvent des **NESTED LOOPS** et des **index** pour afficher rapidement les premières lignes.

102

Hints sous Oracle

ALTER SESSION SET OPTIMIZER_GOAL

- **CHOOSE**
  - Mode **automatique** : Oracle décide.
  - S'il y a des statistiques, il utilise le mode **ALL_ROWS** ou **FIRST_ROWS** selon le contexte.
  - S'il n'y a pas de statistiques, il utilise le mode **RULE**.
  - Ce mode est aussi considéré comme **désuet**, car il repose sur la présence ou non de statistiques.

Exemple:

```
ALTER SESSION SET OPTIMIZER_GOAL = CHOOSE;

SELECT *
FROM products
WHERE category_id = 2;
```

- S'il y a des statistiques → Oracle utilise le **Cost-Based Optimizer**.
- Sinon → il bascule sur le **Rule-Based Optimizer**.

103

Hints sous Oracle

- **ORDERED**
  - Oracle **respecte l'ordre des tables** dans la clause FROM pour effectuer les jointures.
  - Par défaut, Oracle décide lui-même de l'ordre optimal, mais ce hint oblige à **suivre l'ordre d'apparition**.
  - Utile pour **expérimenter manuellement** si un certain ordre de jointure améliore les performances.

- Exemple

```
SELECT /*+ ORDERED */ e.name, d.name
FROM departments d, employees e
WHERE d.id = e.dept_id;
```

Hint	Effet
`ORDERED`	Oracle suit l'ordre des tables dans `FROM`
`FULL(t)`	Forcer un **scan complet** de la table `t`
`INDEX(t, i)`	Forcer l'utilisation de l'index `i` sur la table `t`

104

Hints sous Oracle

- **FULL(table_name)**
  - Force un **scan complet de la table** (Full Table Scan), même si un index est disponible.
  - Utile quand on sait que lire toute la table est plus rapide que d'utiliser un index (ex : table petite, ou requête sur une grande portion de données).

- **Exemple**

```
SELECT /*+ FULL(employees) */ * FROM employees WHERE department_id = 10;
```

Hint	Effet
`ORDERED`	Oracle suit l'ordre des tables dans `FROM`
`FULL(t)`	Forcer un **scan complet** de la table `t`
`INDEX(t, i)`	Forcer l'utilisation de l'index `i` sur la table `t`

105

Hints sous Oracle

- **INDEX(table_name index_name)**
  - Force Oracle à **utiliser un index spécifique** sur une table.
  - Pratique quand l'optimiseur ne choisit pas l'index qu'on juge le plus pertinent (ex : mauvaise estimation de coûts).

- **Exemple**

```
SELECT /*+ INDEX(employees emp_idx_name) */ *
FROM employees
WHERE name = 'Mohamed';
```

Hint	Effet
`ORDERED`	Oracle suit l'ordre des tables dans `FROM`
`FULL(t)`	Forcer un **scan complet** de la table `t`
`INDEX(t, i)`	Forcer l'utilisation de l'index `i` sur la table `t`

106

Contrôle du processus d'optimisation- Cas Oracle

Explain plan

- Créer et sauvegarder un plan d'exécution
- Pour créer un plan d'exécution, il faut préfixer la requête SQL respective
  - avec explain plan for :

```
EXPLAIN PLAN FOR select * from table_T
```

  - Pour afficher le plan d'exécution, il faut exécuter

```
SELECT * FROM table(dbms_xplan.display);
```

  - La requête affichera le plan d'exécution comme

```
-----------------------------------
| Id | Operation          | Name | Rows | Bytes | Cost (%CPU)|
-----------------------------------
| 0 | SELECT STATEMENT |      |    1 |    2 |    2   (0)|
| 1 | TABLE ACCESS FULL| DUAL |    1 |    2 |    2   (0)|
-----------------------------------
```

Contrôle du processus d'optimisation- Cas Oracle

Explain plan

- Accès aux tables et aux index
  - **INDEX UNIQUE SCAN**
    - L'opération **INDEX UNIQUE SCAN** réalise un parcours seul du B-tree. La base de données utilise cette opération si une contrainte unique assure que le critère de recherche correspondra à pas plus d'une entrée.
  - **INDEX RANGE SCAN**
    - L'opération **INDEX RANGE SCAN** réalise un parcours du B-tree *et* suit la chaîne de nœuds feuilles pour trouver toutes les entrées correspondantes.

Les prédicats de filtre de l'index causent souvent des problèmes de performance pour un INDEX RANGE SCAN.

Contrôle du processus d'optimisation- Cas Oracle

Explain plan

- Accès aux tables et aux index
  - **INDEX FULL SCAN**
    - Lit l'index entier dans l'ordre de l'index.
    - Suivant différentes statistiques système, la base de données peut choisir de réaliser cette opération car elle a besoin de toutes les lignes dans l'ordre de l'index, par exemple à cause d'une clause order by.
    - À la place, l'optimiseur pourrait aussi utiliser un **INDEX FAST FULL SCAN** et réaliser une opération de tri supplémentaire.

Contrôle du processus d'optimisation- Cas Oracle

Explain plan

- Accès aux tables et aux index
  - **INDEX FAST FULL SCAN**
    - Lit l'index en entier (toutes les lignes) comme stocké sur disque.
    - Cette opération est typiquement exécutée à la place d'un parcours complet de table si toutes les colonnes requises sont disponibles dans l'index.
    - Comme le TABLE ACCESS FULL, l'opération INDEX FAST FULL SCAN peut bénéficier d'opérations de lecture multi-blocs

110

Contrôle du processus d'optimisation- Cas Oracle

Explain plan

- Accès aux tables et aux index
  - **TABLE ACCESS BY INDEX ROWID**
    - Récupère une ligne à partir de la table en utilisant le ROWID récupéré lors d'une recherche précédente dans l'index.
  - **TABLE ACCESS FULL**
    - Cette opération est aussi connue sous le nom de parcours complet de table.
    - Elle lit la table entière, toutes les lignes et toutes les colonnes, comme elle est stockée sur le disque.
    - Bien que les opérations multi-blocs accélèrent fortement la rapidité d'un parcours complet de table, c'est l'une des opérations les plus coûteuses.
    - En dehors de gros taux d'entrées/sorties, un parcours complet de table doit inspecter toutes les lignes de la table, donc il peut aussi consommer beaucoup de temps processeur.

111

Contrôle du processus d'optimisation- Cas Oracle

- Généralement, les opérations de jointure traitent deux tables à la fois. Au cas où la requête a plus de jointures, elles sont exécutées séquentiellement : tout d'abord deux tables, puis le résultat intermédiaire et la table suivante.
- Dans le contexte des jointures, le terme « table » peut aussi signifier « résultat intermédiaire ».
  - **NESTED LOOPS JOIN**
    - Joint deux tables en récupérant le résultat d'une table et en recherchant la condition de jointure dans l'autre table ligne par ligne.
  - **HASH JOIN**
    - La jointure de hachage charge les enregistrements candidats d'un côté de la jointure dans une table de hachage qui est ensuite sondée pour chaque ligne de l'autre côté de la jointure.
  - **MERGE JOIN**
    - La jointure d'assemblage combine deux listes triées comme une fermeture Éclair. Les deux côtés de la jointure doivent être pré-triés.

112

Contrôle du processus d'optimisation- Cas Oracle

- **Explain plan**
  - Tri et regroupement
    - **SORT ORDER BY**
      - Trie le résultat suivant la clause **order by**.
    - **SORT ORDER BY STOPKEY**
      - Trie un sous-ensemble du résultat suivant la clause **order by.**
      - Utilisé pour les requêtes top-N si une exécution en pipeline n'est pas possible.
    - **SORT GROUP BY**
      - Trie l'ensemble de résultat sur les colonnes de la clause **group by** et agrège le résultat trié dans une deuxième étape.

Contrôle du processus d'optimisation- Cas Oracle

- **Explain plan**
  - Tri et regroupement
    - **SORT GROUP BY NOSORT**
      - Agrège un ensemble pré-trié suivant la clause group by.
      - Cette opération ne place pas le résultat intermédiaire dans un tampon : il est exécuté dans un pipeline.
    - **HASH GROUP BY**
      - Groupe les résultats suivant une table de hachage.
      - Cette opération a besoin de grandes quantités de mémoire pour matérialiser le résultat intermédiaire (sans pipeline).
      - La sortie n'est pas ordonnée de quelle que façon que ce soit.

Contrôle du processus d'optimisation- Cas Oracle

- **La base de données Oracle utilise trois méthodes différentes pour appliquer les clauses where (prédicats) :**
  - **Prédicats d'accès (« access »)**
    - Les prédicats d'accès expriment les conditions de début et de fin des parcours de nœuds feuilles.
  - **Prédicat de filtre d'index (« filter » pour les opérations sur les index)**
    - Les prédicats de filtres d'index sont appliqués seulement lors du parcours de nœuds feuilles.
    - Ils ne contribuent pas aux conditions de début et d'arrêt, et ne diminuent pas l'intervalle parcouru.
  - **Prédicat de filtre au niveau table (« filter » pour les opérations sur la table)**
    - Les prédicats sur les colonnes qui ne font pas partie de l'index sont évalués au niveau de la table.
    - Pour cela, la base de données doit tout d'abord charger la ligne à partir de la table.

References

- A. Silberschatz, H. Korth and S. Sudarshan, *Database System Concepts* (Sixth Edition), McGraw-Hill, 2010
- David Taniar, Clement H. C. Leung, Wenny Rahayu, Sushant Goel. High Performance Parallel Database Processing and Grid Databases – 2008
- Lightstone & Teorey & Nadeau. Physical Database Design, the database professional's guide to exploiting indexes, views, storage, and more – 2007
- Georges Gardarin . Bases de données – 2003
- http://use-the-index-luke.com/fr/sql/plans-dexecution/oracle/predicats-acces-filtre

4. Jointure avec index

- Dans le cas ou l'une des relations est indexée sur l'attribut de jointure par exemple R) dans ce cas il suffit de balayer la deuxième relation S et accéder au fichier index pour chaque tuple.
- En cas de succès, on procède à la concaténation pour composer le tuple résultat.
- Lorsque les deux relations sont indexée sur les attributs de jointure il suffit de fusionner les deux index, les couples des adresses des tuples concaténés sont alors obtenus.

- $R \bowtie_{a=b} S$
- Comme boucles imbriquées mais avec l'utilisation de l'index sur l'attribut b de S

1. charger une partie de R en RAM
2. Charger l'index
3. Pour chaque tuple de R, parcourir l'index, retrouver les RID des tuples de S à joindre
4. Charger ces tuples en RAM
5. Coller les tuples

Coût = | index | + | R | + || R || * | S | / NDIST (S, b)

Comment se fait l'optimisation

- Chercher le meilleur plan d 'exécution?
  - coût excessif
- Solution approchée à un coût raisonnable
  - Générer les alternatives
    - heuristiques
- Choisir la meilleure estimation approximative du coût

119

Exemple

- Soit le schema de la BD: **Departments** (id , name)
  Employees (id , name , *dept_id**)

```
EXPLAIN PLAN FOR
SELECT e.name, d.name
FROM employees e , departments d
WHERE e.dept_id = d.id;
```

ID	Operation	Object	Rows	Cost
0	SELECT STATEMENT			4
1	NESTED LOOPS		3	4
2	TABLE ACCESS FULL	DEPARTMENTS	2	2
3	TABLE ACCESS FULL	EMPLOYEES	3	2

- Oracle a utilisé une **jointure en NESTED LOOPS**, car les tables sont petites.
- Il n'y a pas d'index spécifique, donc Oracle fait un **FULL SCAN** des deux tables.
- Bonne performance pour un petit dataset.

120

```
CREATE INDEX idx_emp_dept ON employees(dept_id);

EXPLAIN PLAN FOR
SELECT e.name, d.name
FROM employees e, departments d
WHERE e.dept_id = d.id;
```

ID	Operation	Object	Rows	Cost
0	SELECT STATEMENT			3
1	NESTED LOOPS		3	3
2	TABLE ACCESS FULL	DEPARTMENTS	2	2
3	INDEX RANGE SCAN	IDX_EMP_DEPT	3	1

- L'optimiseur a détecté l'**index idx_emp_dept** et l'a utilisé pour accéder plus rapidement aux lignes employees.
- Le coût est plus bas qu'avant.
- Toujours en NESTED LOOPS mais **optimisé grâce à l'index**.

121